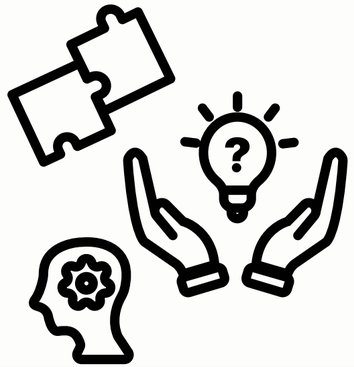
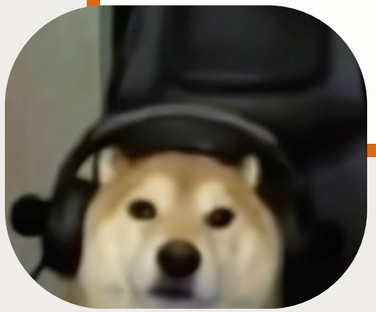
Ayudantía 9 · Big Data · Recuperación de Información

REPRESENTACIÓN DE DATOS

Roberto Ibaceta Cisternas

ÍNDICE





01. Que veremos hoy

02. Limpieza y procesamiento previo

03. Conjuntos

04. Vectores dispersos: TF-IDF

05. Vectores densos

07. Errores típicos y cierre



Recuperación de Información: dos fases



I. Fase offline

- Recolección
- **Limpieza**
- **Procesamientos previos**
- Indexado
- **Traducción a otra representación**

II. Fase online

- Procesamiento de la consulta
- Recuperación y entrega de resultados

La consulta pasa por la misma traducción que los documentos, solo que en esta fase.

Usuario → Q: «perro» → **Traducción a la representación matemática** → Comparación o medición contra los datos → Ordenar resultados por relevancia y entregar

Los pilares

Representación

¿Cómo traducimos?

- **Conjuntos** (de palabras): conjuntos booleanos
- **Vectores dispersos:** asignar peso a palabras
- **Vectores densos:** traducción geométrica

Medida de similitud

¿Cómo comparamos?

- Intersección de conjuntos
- Distancia espacial
- Alineación direccional
- Probabilidades

Optimización

¿Cómo hacerlo eficiente?

- Agrupación espacial
- Funciones hash

Hoy: un bloque por cada forma de representación. Similitud y optimización son la próxima práctica.



Lo que vieron en clases

Doc	Texto original
d_1	Spark procesa los datos en memoria.
d_2	Hadoop procesa datos y más datos en disco.
d_3	La consulta de datos con índices.
d_4	Un índice en MEMORIA: consultas rápidas, CONSULTAS de datos.

Notación de la cátedra

D → conjunto de documentos, el «corpus». **d** → un documento ($d \in D$). **t** → término o palabra de interés. **N = |D| = 4**.

El problema

Para el computador «MEMORIA» y «memoria» son distintas, y «consulta» no tiene relación con «consultas». Antes de traducir hay que limpiar.

Pasos 1 y 2: normalizar y tokenizar

Antes (d_4)

Un índice en MEMORIA: consultas rápidas, CONSULTAS de datos.

Después: 9 tokens

`un` `indice` `en` `memoria` `consultas` `rapidas` `consultas` `de` `datos`

Todo a minúsculas

MEMORIA → memoria

Fuera las tildes

índice → indice

Fuera la puntuación

Se separa por espacios: cada pedazo es un token

Quitar tildes tiene costo: «más» y «mas» pasan a ser lo mismo.

Paso 3: stopwords

Lista comunes

`el` `la` `los` `las` `un` `una`
`de` `en` `y` `con` `mas`

d_4 después de quitarlas

~~`un`~~ `indice` ~~`en`~~ `memoria` `consultas` `rapidas`
`consultas` ~~`de`~~ `datos`

Quedan 6 de 9 tokens. Los tachados son stopwords.

> Son palabras que aparecen en casi todo texto y no ayudan a distinguir documentos.
>
> **La lista es una decisión de diseño:** no existe una lista oficial.





Paso 4: stemming y lematización

Stemming

Corta sufijos con reglas. Es rápido, y el resultado puede no ser una palabra.

`consultas` → **`consult`**

`índices` → **`indic`**

`rápidas` → **`rap`**

Lematización

Usa un diccionario y devuelve la forma base. Es más lenta y más precisa.

`consultas` → **`consulta`**

`índices` → **`índice`**

`rápidas` → **`rápido`**

El stem es una etiqueta para agrupar variantes: «consulta» y «consultas» pasan a ser el mismo término.

Representación 1: conjuntos

El ejemplo de la cátedra

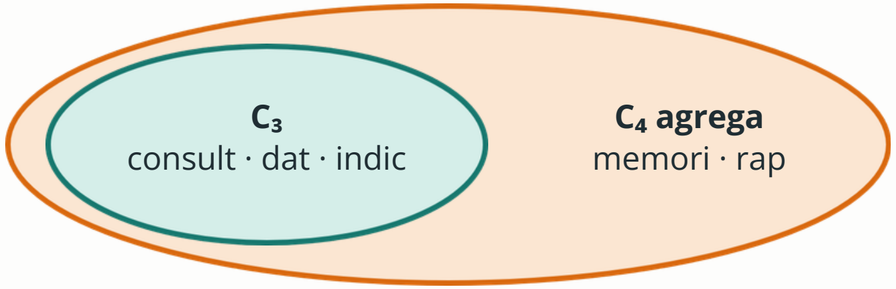
Q_1 = «vectores dispersos»
Q_2 = «vectores densos»
C_1 = {vec, disp}
C_2 = {vec, densos}

$C_1 \cap C_2$ = {vec}

Comparar dos conjuntos es mirar qué términos comparten.

Con nuestro ejemplo

C_1 = {dat, memori, proces, spark}
C_2 = {dat, disc, hadoop, proces}
C_3 = {consult, dat, indic}
C_4 = {consult, dat, indic, memori, rap}



$C_3 \cap C_4$ = {consult, dat, indic} · Jaccard: $J(C_3, C_4) = 3 / 5 = 0{,}6$

Un conjunto solo responde si el término está o no está: sin repeticiones ni orden. d_2 dice «datos» dos veces y en C_2 aparece una.

Del conjunto al vector binario

Término t	d_1	d_2	d_3	d_4
consult	0	0	1	1
dat	1	1	1	1
disc	0	1	0	0
hadoop	0	1	0	0
indic	0	0	1	1
memori	1	0	0	1
proces	1	1	0	0
rap	0	0	0	1
spark	1	0	0	0



Lo que hizo el profesor

V_1 = (1, 1, 0)
V_2 = (1, 0, 1)

Se fija un orden para los términos. Cada posición vale 1 si el término está y 0 si no.

Con d_3 (columna naranja)

C_3 = {consult, dat, indic}
V_3 = (1, 1, 0, 0, 1, 0, 0, 0, 0)

Conjunto y vector binario guardan la misma información. El vector permite hacer cuentas.

Esta tabla se llama matriz de incidencia: 9 términos por 4 documentos.

Representación 2: asignar peso con TF

TF(t, d)	d_1	d_2	d_3	d_4
consult	0	0	1	2 ←
dat	1	2 ←	1	1
disc	0	1	0	0
hadoop	0	1	0	0
indic	0	0	1	1
memori	1	0	0	1
proces	1	1	0	0
rap	0	0	0	1
spark	1	0	0	0

TF: frecuencia del término

`TF(t, d) = veces que t aparece en d`

El TF crece con la frecuencia del término. Respecto del vector binario cambian dos celdas (marcadas con ←).

Bolsa de palabras: se pierde el orden. «Spark procesa datos» y «datos procesa Spark» quedan iguales.

El problema de contar

La fila «dat» es 1, 2, 1, 1. Es el término más frecuente del corpus y el más inútil para buscar, porque está en todos los documentos.

IDF: frecuencia inversa de documentos

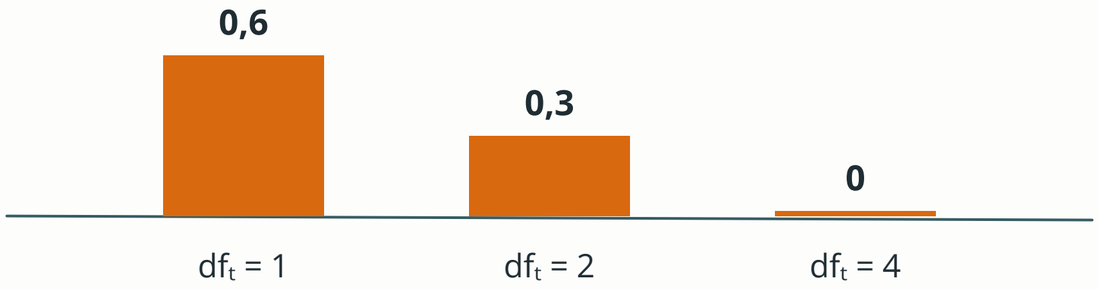
$$IDF(t, D) = \log_{10}(N / df_t)$$

Notación de la cátedra

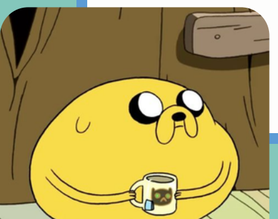
D → conjunto de documentos, «corpus»
d → un documento del corpus (d ∈ D)
t → término o palabra de interés
N = |D| → en nuestro corpus, N = 4
df_t → cantidad de documentos que contienen t

df_t	N / df_t	IDF(t, D)
1	4	$\log_{10} 4 \approx 0{,}6$
2	2	$\log_{10} 2 \approx 0{,}3$
4	1	$\log_{10} 1 = 0$

IDF según en cuántos documentos aparece el término



El IDF decrece mientras el término aparezca en más documentos. Dato: $\log_{10} 2 \approx 0{,}3$ y $\log_{10} 4$ es el doble.



TF-IDF: ejemplo resuelto con d_4

$$TF\text{-}IDF(t, d, D) = TF(t, d) \cdot IDF(t, D)$$

Término t	TF(t, d_4)	IDF(t, D)	TF-IDF
consult	2	0,3	2 · 0,3 = 0,6
dat	1	0	1 · 0 = 0
indic	1	0,3	1 · 0,3 = 0,3
memori	1	0,3	1 · 0,3 = 0,3
rap	1	0,6	1 · 0,6 = 0,6

Cuándo el peso es alto

Cuando el término se repite en el documento **y** es raro en el corpus.

«consult» y «rap» pesan lo mismo: uno es frecuente en d_4; el otro aparece una vez, pero solo en este documento.

«dat» pesa 0: está en los 4 documentos y no distingue a ninguno.

¿Por qué se llaman vectores dispersos?

Vector	consult	dat	disc	hadoop	indic	memori	proces	rap	spark
d_3	0,3	0	0	0	0,3	0	0	0	0
Q	0,3	0	0	0	0	0,3	0	0,6	0

Q = «consultas rápidas en memoria». La consulta se traduce con el mismo procedimiento que los documentos: limpiar, contar y multiplicar por el IDF.

36
celdas en la matriz

24
son ceros

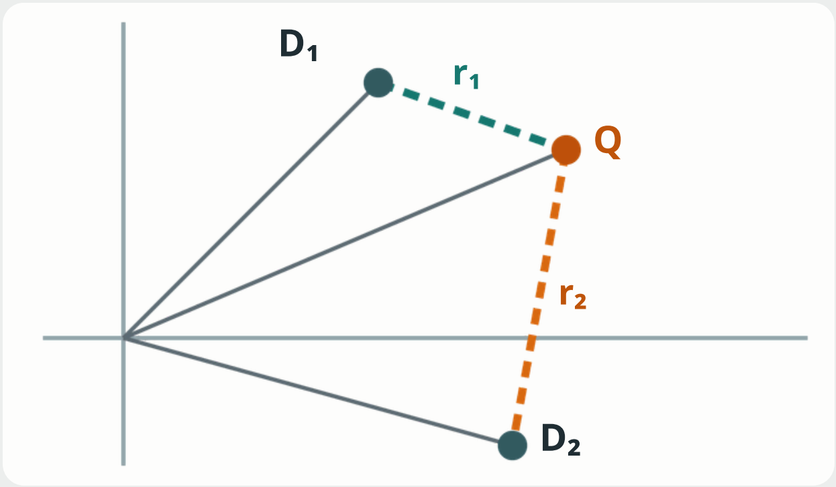
Hay una dimensión por cada término del vocabulario.

Con un vocabulario real de decenas de miles de términos casi todo es cero: por eso se guardan solo los valores distintos de cero.

Representación 3: vectores densos

El ejemplo de la cátedra

D_1 = libro de cálculo · **D_2** = etiqueta de un pan integral **Q = «integral»** · ¿Cuál debería ser el orden de relevancia?



$r_1 < r_2 \rightarrow D_1$ es más similar a Q

Vector disperso

Una dimensión por término y casi todo ceros. Solo ve que la palabra «integral» está en ambos documentos, no en qué sentido se usa.

Vector denso: traducción geométrica

Pocas dimensiones, todas con valor. Cada texto es un punto, y los textos de significado parecido quedan cerca.

Hoy basta saber. Cómo se construyen esos vectores es materia de mas adelante.

4 errores típicos

1. Confundir TF con df_t

TF: veces que el término aparece en el documento. df_t: cantidad de documentos que contienen el término.

2. Usar como N el número de palabras

N = |D| es la cantidad de documentos del corpus. En esta clase, N = 4.

3. No traducir la consulta

Q pasa por los mismos pasos que los documentos: limpiar, contar y multiplicar por el IDF.

4. Creer que IDF = 0 es un error

Un término que está en todos los documentos no distingue a ninguno. Por eso pesa 0.

En una prueba: declara la fórmula que usas, $IDF(t, D) = \log_{10}(N / df_t)$, y los logaritmos que te entregan.

3 ideas para llevarse

1. **Conjuntos: ¿está o no está?** Sirven para filtrar con AND, OR y NOT, pero **no ordenan por relevancia**.

2. **Vectores dispersos: ¿cuánto pesa?** El TF crece con la frecuencia del término; el IDF decrece si aparece en más documentos. Juntos son el **TF-IDF**.

3. **Vectores densos: ¿dónde queda en el espacio?** Los textos de **significado parecido** quedan cerca.



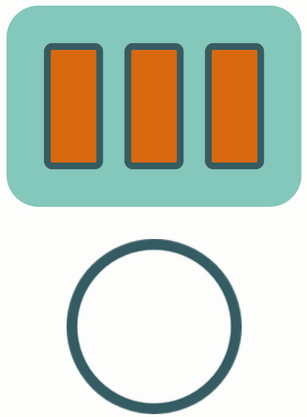
C O N T A C T O

Gmail
roberto.ibaceta@mail.udp.cl

Wsp
+569 42663436

Ds
roberibaz





MUCHAS GRACIAS

